

编译器设计实验-第一次实验

Haochen You

April 2026

1 实验任务（必做）

1.1 任务描述

1. 配置编译器能运行的环境；
2. 写一个简单的程序验证；
3. 用 C++ 实现 DFA 的模拟程序。

1.2 环境配置

首先，我们在阿里云的 PAI-DSW 中认证授权并开通服务，系统会自动创建 OSS Bucket 并绑定到 PAI 默认工作空间。接下来我们创建 DSW 实例，结果如下：



图 1: 创建 DSW 实例

其中，处理器我们选择 ecs.g7.large，镜像我们选择 python:3.10.19-cpu-ubuntu22.04。网页 IDE 如下：

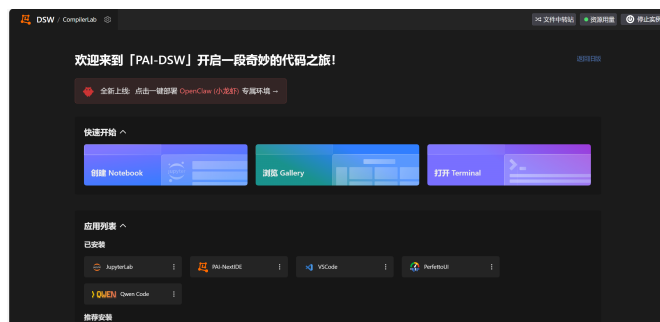


图 2: Web IDE

之后，我们验证环境配置。执行 `sudo apt install -y build-essential` 下载 g++，发现已经有相关配置了。

```
root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# sudo apt install -y build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.9ubuntu3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@dsw-761783-859546dc4d-8scgt:/mnt/workspace#
```

图 3: g++ environment

1.3 简单测试

我们编写一个简单的测试 HelloWorld 程序，存储在 `/mnt/workspace/main.cpp` 之下，测试一下程序在实例中能否正常运行，结果如下：

```
root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# gcc -o main main.cpp
root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# ./main
Hello World!
```

图 4: Test

证明当前环境可以正常运行 C++ 程序，接下来就可以模拟 DFA 的实现了。

1.4 DFA 模拟程序实现

接下来，我们实现一个 DFA 的模拟程序，其功能包括读取输入（字符集、状态集、开始状态、接受状态、状态转换表），检查 DFA 是否合法，输出长度小于某个值的所有合法串，规则判定。

首先我们实现 DFA 的读入，代码框架如下：

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <set>
5 using namespace std;
6 struct DFA{
7     string alphabet;
8     int stateCount;
9     set<int> states;
10    int startState;
11    set<int> acceptStates;
12    vector<vector<int>> transition;
13 };
14 DFA readDFA()
15 {
16     DFA dfa;
17     freopen("dfa.txt","r",stdin);
18     cin>>dfa.alphabet;
19     string line;
20     getline(cin, line);
21     getline(cin, line);
22     for (char c : line)
23     {
```

```

24     if (c >= '0' && c <= '9')
25     {
26         dfa.states.insert(c - '0');
27     }
28 }
29 dfa.stateCount = dfa.states.size();
30 cin>>dfa.startState;
31 getline(cin, line);
32 getline(cin, line);
33 for (char c : line)
34 {
35     if (c >= '0' && c <= '9')
36     {
37         dfa.acceptStates.insert(c - '0');
38     }
39 }
40 dfa.transition.resize(dfa.stateCount + 1,
41     vector<int>(dfa.alphabet.size()));
42 for (int i = 1; i <= dfa.stateCount; i++)
43 {
44     for (int j = 0; j < dfa.alphabet.size(); j++)
45     {
46         cin >> dfa.transition[i][j];
47     }
48 }
49 return dfa;
50 }
51 int main()
52 {
53     DFA dfa = readDFA();
54     cout << "字符集: " << dfa.alphabet << endl;
55     cout << "状态集: ";
56     for (int s : dfa.states) cout << s << " ";
57     cout << endl;
58     cout << "开始状态: " << dfa.startState << endl;
59     cout << "接受状态: ";
60     for (int s : dfa.acceptStates) cout << s << " ";
61     cout << endl;
62     return 0;
63 }

```

上面的程序定义了一个 DFA 结构体，包含字符集、状态集、开始状态、接受状态集和转移函数五个字段，对应 DFA 的五元组定义。

readDFA() 函数通过 freopen 依次读取各字段：字符集以字符串形式读入；状态集和接受状态集以整数集合形式逐字符读入 set；状态转换表以二维数组形式存储，第一维为当前状态，第二维为输入字符在字母表中的下标，值为转移后的目标状态。

主函数调用 readDFA() 完成文件读取后，将 DFA 各字段打印输出，验证正确性，结果如下：

```

root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# g++ -o main main.cpp
root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# ./main
字符集: ab
状态集: 1 2 3 4
开始状态: 1
接受状态: 4

```

图 5: Result

完成读入之后，我们实现检查 DFA 是否合法的功能。增加的 checkDFA() 函数如下：

```

1 bool checkDFA(DFA dfa)
2 {
3     bool valid = true;
4     if (dfa.states.find(dfa.startState) == dfa.states.end())
5     {
6         cout << "错误: 开始状态 " << dfa.startState << " 不在状态集中!" << endl;
7         valid = false;
8     }
9     else cout << "开始状态合法" << endl;
10
11     if (dfa.acceptStates.empty())
12     {
13         cout << "错误: 接受状态集为空!" << endl;
14         valid = false;
15     }
16     else cout << "接受状态集非空" << endl;
17
18     for (int s : dfa.acceptStates)
19     {
20         if (dfa.states.find(s) == dfa.states.end())
21         {
22             cout << "错误: 接受状态 " << s << " 不在状态集中!" << endl;
23             valid = false;
24         }
25     }
26     if (valid) cout << "接受状态集合法" << endl;
27     if (valid) cout << "\nDFA 合法, 可以运行!" << endl;
28     else cout << "\nDFA 不合法, 请检查输入文件!" << endl;
29     return valid;
30 }

```

checkDFA() 函数一共包含三项检查：首先验证开始状态是否存在于状态集中；其次检查接受状态集是否非空，保证 DFA 存在可接受的终止状态；最后逐一验证接受状态集中的每个状态是否都属于状态集，避免出现未定义状态。任意一项检查不通过则输出错误信息并返回 false，全部通过则返回 true。

测试之前的 DFA 结果：

```

root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# g++ -o main main.cpp
root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# ./main
字符集: ab
状态集: 1 2 3 4
开始状态: 1
接受状态: 4
开始状态合法
接受状态集非空
接受状态集合法
DFA 合法, 可以运行!

```

图 6: Test

下面我们来写判定函数，用于判定某个串是否能被 DFA 接受，代码如下：

```

1 bool recognize(DFA dfa, string s)
2 {
3     int cur = dfa.startState;
4     for (char c : s)
5     {
6         int idx = dfa.alphabet.find(c);
7         if (idx == string::npos)
8         {
9             cout << "错误: 字符 " << c << " 不在字符集中" << endl;
10            return false;
11        }
12        cur = dfa.transition[cur][idx];
13    }
14    return dfa.acceptStates.count(cur) > 0;
15 }

```

recognize() 函数模拟 DFA 运行过程，函数从开始状态出发，逐个读取字符串中的每个字符，首先在字母表中查找该字符的下标，若字符不在字母表中则直接返回非法；否则根据状态转换表进行状态转移，更新当前状态，最后判断最终所处状态是否属于接受状态集。

测试结果：

```

root@dsw-761783-859546dc4d-8scgt:/mnt/workspace# ./main
字符集: ab
状态集: 1 2 3 4
开始状态: 1
接受状态: 4
开始状态合法
接受状态集非空
接受状态集合法
DFA 合法, 可以运行!
请输入要判定的字符串: aab
aab 是合法字符串 (被DFA接受)

```

图 7: Test

最后，我们实现功能：输出长度小于某个值的所有合法串。函数如下：

```

1 void enumerate(DFA dfa, int maxLen)

```

```

2 {
3     vector<string> queue;
4     queue.push_back("");
5     cout << "DFA接受的长度<=" << maxLen << "的所有字符串：" << endl;
6     for (int i = 0; i < queue.size(); i++) {
7         string cur = queue[i];
8         if (cur.length() > 0 && recognize(dfa, cur))
9             {
10                 cout << cur << endl;
11             }
12         if ((int)cur.length() < maxLen)
13             {
14                 for (char c : dfa.alphabet)
15                     {
16                         queue.push_back(cur + c);
17                     }
18             }
19     }
20 }

```

enumerate() 函数采用 BFS 的思路，从空串开始逐步扩展所有可能的字符串。每次取出当前字符串，若长度大于 0 则调用 recognize() 函数判断是否被 DFA 接受，若接受则输出；若当前字符串长度未达到上限 N，则将其与字母表中每个字符拼接后加入队列。最终输出所有长度小于等于 N 且能被 DFA 接受的合法字符串。

测试结果如下：

```

请输入最大长度N: 3
DFA接受的长度<=3的所有字符串:
aa
bb
aaa
aab
abb
baa
bba
bbb

```

图 8: Test

2 个人总结

本次实验在阿里云 PAI-DSW 平台上，使用 C++ 语言完成了 DFA 模拟程序的设计与实现。我设计了 txt 格式存储 DFA，通过 freopen 读入 DFA 文件，使用结构体 DFA 依次读取字符集、状态集、开始状态、接

受状态集和转移函数。在此基础上实现了 `checkDFA()` 函数，验证 DFA 的合法性，确保开始状态唯一且在状态集中、接受状态集非空且包含于状态集中。接着实现了 `recognize()` 函数，模拟 DFA 从开始状态出发逐字符进行状态转移的运行过程，判定输入字符串是否被接受。最后实现了 `enumerate()` 函数，采用 BFS 枚举所有长度不超过 N 的字符串，并调用判定函数筛选输出合法字符串。经过本次实验，我加深了对 DFA 五元组定义及其运行原理的理解，掌握了在云端环境下进行 C++ 程序开发与调试的基本流程。